Create Restfull API in respoitory pattern | Standard API development

Shiva Gautam - April 21, 2026

Here's a **clean, modern implementation of a RESTful API in ASP.NET Core Web API using the Repository Pattern** (aligned with .NET 8/9 style).

Step-by-Step: ASP.NET Core Web API + Repository Pattern

📦 What is Repository Pattern?

In simple terms:

👉 It acts as a **middle layer between your Controller and Database**

Instead of writing database queries directly in controllers, you use a **repository** to handle all data operations.

Why Use Repository Pattern?

1. Clean Code

Keeps your controllers simple and focused on API logic.

2. Separation of Concerns

- Controller → Handles HTTP requests
- Repository → Handles database logic

3. Easy Testing

You can mock repositories during unit testing.

4. Flexibility

If you switch from SQL Server → MongoDB, only repository changes.

Without Repository (Bad Practice)

```
public IActionResult Get()
{
    var data = _context.Products.ToList(); // Direct DB access ❌
    return Ok(data);
}
```

👉 Problem:

- Tight coupling with database
- Hard to test

- Messy code

✅ With Repository (Good Practice)

```
public IActionResult Get()
{
    var data = _repo.GetAll(); // Clean abstraction ✅
    return Ok(data);
}
```

👉 Benefit:

- Clean, readable, maintainable

🏗️ Structure Overview

```
Controller → Repository → DbContext → Database
```

🔄 Flow of Data

1. Client hits API (GET /products)
2. Controller receives request
3. Controller calls Repository
4. Repository talks to Database
5. Data returns back to Controller
6. Controller sends response

🧩 Types of Repository

1. Generic Repository

Reusable for all entities

```
IRepository<T>
```

2. Specific Repository

For specific entity (like Product)

```
IProductRepository
```

⚠️ When NOT to Use It

Don't blindly use repository pattern if:

- Your project is very small
- You are already using **Entity Framework Core** (it already acts like a repository)

👉 Overusing it can create unnecessary complexity.

🧠 Important Concept

Repository Pattern is often used with:

- Unit of Work Pattern
- Dependency Injection
- Clean Architecture

SHIVA TUTORIAL

Tutorials
C#
ASP.NET CORE MVC
PYTHON
JAVA
JAVASCRIPT
REACT JS
NODE JS
Salesforce
DEVOPS
Software Testing
Selenium JAVA
Selenium Python
Spring MVC & Boot
Angular 21.0

Classroom Training
Online Training

🔥 Short Definition (Interview Ready)

👉 *"Repository Pattern is a design pattern that abstracts data access logic and provides a clean separation between business logic and data layer."*

1️⃣ Create Project

```
dotnet new webapi -n RepoPatternAPI
cd RepoPatternAPI
```

2️⃣ Create Model (Entity)

📁 **Models/Product.cs**

```
namespace RepoPatternAPI.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

3️⃣ Setup DbContext

📁 **Data/AppDbContext.cs**

```
using Microsoft.EntityFrameworkCore;
using RepoPatternAPI.Models;

namespace RepoPatternAPI.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }

        public DbSet<Product> Products { get; set; }
    }
}
```

4️⃣ Create Repository Interface

📁 **Repositories/IProductRepository.cs**

```
using RepoPatternAPI.Models;

namespace RepoPatternAPI.Repositories
{
    public interface IProductRepository
    {
        Task<IEnumerable<Product>> GetAllAsync();
        Task<Product?> GetByIdAsync(int id);
        Task AddAsync(Product product);
        Task UpdateAsync(Product product);
        Task DeleteAsync(int id);
    }
}
```

5️⃣ Implement Repository

📁 **Repositories/ProductRepository.cs**

SHIVA TUTORIAL

Tutorials
C#
ASP.NET CORE MVC
PYTHON
JAVA
JAVASCRIPT
REACT JS
NODE JS
Salesforce
DEVOPS
Software Testing
Selenium JAVA
Selenium Python
Spring MVC & Boot
Angular 21.0

Classroom Training
Online Training

```
using Microsoft.EntityFrameworkCore;
using RepoPatternAPI.Data;
using RepoPatternAPI.Models;

namespace RepoPatternAPI.Repositories
{
    public class ProductRepository : IProductRepository
    {
        private readonly AppDbContext _context;

        public ProductRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<Product>> GetAllAsync()
        {
            return await _context.Products.ToListAsync();
        }

        public async Task<Product?> GetByIdAsync(int id)
        {
            return await _context.Products.FindAsync(id);
        }

        public async Task AddAsync(Product product)
        {
            await _context.Products.AddAsync(product);
            await _context.SaveChangesAsync();
        }

        public async Task UpdateAsync(Product product)
        {
            _context.Products.Update(product);
            await _context.SaveChangesAsync();
        }

        public async Task DeleteAsync(int id)
        {
            var product = await GetByIdAsync(id);
            if (product != null)
            {
                _context.Products.Remove(product);
                await _context.SaveChangesAsync();
            }
        }
    }
}
```

6️⃣ Register Services (Dependency Injection)

📁 **Program.cs**

```
using Microsoft.EntityFrameworkCore;
using RepoPatternAPI.Data;
using RepoPatternAPI.Repositories;

var builder = WebApplication.CreateBuilder(args);

// Add DbContext
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("TestDb")); // or SQL Server

// Register Repository
builder.Services.AddScoped<IProductRepository, ProductRepository>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();
```

```
app.Run();
```

7 Create Controller (REST API)

Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using RepoPatternAPI.Models;
using RepoPatternAPI.Repositories;

namespace RepoPatternAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private readonly IProductRepository _repo;

        public ProductController(IProductRepository repo)
        {
            _repo = repo;
        }

        // GET: api/product
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            var data = await _repo.GetAllAsync();
            return Ok(data);
        }

        // GET: api/product/5
        [HttpGet("{id}")]
        public async Task<IActionResult> GetById(int id)
        {
            var product = await _repo.GetByIdAsync(id);
            if (product == null) return NotFound();
            return Ok(product);
        }

        // POST: api/product
        [HttpPost]
        public async Task<IActionResult> Create(Product product)
        {
            await _repo.AddAsync(product);
            return Ok(product);
        }

        // PUT: api/product/5
        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, Product product)
        {
            if (id != product.Id) return BadRequest();

            await _repo.UpdateAsync(product);
            return NoContent();
        }

        // DELETE: api/product/5
        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
        {
            await _repo.DeleteAsync(id);
            return NoContent();
        }
    }
}
```

✅ API Endpoints

Method	Endpoint	Description
GET	`/api/product`	Get all products

Method	Endpoint	Description
GET	/api/product/{id}	Get by ID
POST	/api/product	Create
PUT	/api/product/{id}	Update
DELETE	/api/product/{id}	Delete

Here's a clean, modern implementation of a RESTful API in ASP.NET Core Web API using the Repository Pattern (aligned with .NET 8/9 style).

🚀 Step-by-Step: ASP.NET Core Web API + Repository Pattern

1️⃣ Create Project

```
dotnet new webapi -n RepoPatternAPI
cd RepoPatternAPI
```

2️⃣ Create Model (Entity)

📁 Models/Product.cs

```
namespace RepoPatternAPI.Models
{
    public class Product
    {
        public int Id { get; set; }
        public string Name { get; set; }
        public decimal Price { get; set; }
    }
}
```

3️⃣ Setup DbContext

📁 Data/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using RepoPatternAPI.Models;

namespace RepoPatternAPI.Data
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)
            : base(options) { }

        public DbSet<Product> Products { get; set; }
    }
}
```

4️⃣ Create Repository Interface

📁 Repositories/IProductRepository.cs

```
using RepoPatternAPI.Models;
```

```
namespace RepoPatternAPI.Repositories
{
    public interface IProductRepository
    {
        Task<IEnumerable<Product>> GetAllAsync();
        Task<Product?> GetByIdAsync(int id);
        Task AddAsync(Product product);
        Task UpdateAsync(Product product);
        Task DeleteAsync(int id);
    }
}
```

5 Implement Repository

Repositories/ProductRepository.cs

```
using Microsoft.EntityFrameworkCore;
using RepoPatternAPI.Data;
using RepoPatternAPI.Models;

namespace RepoPatternAPI.Repositories
{
    public class ProductRepository : IProductRepository
    {
        private readonly AppDbContext _context;

        public ProductRepository(AppDbContext context)
        {
            _context = context;
        }

        public async Task<IEnumerable<Product>> GetAllAsync()
        {
            return await _context.Products.ToListAsync();
        }

        public async Task<Product?> GetByIdAsync(int id)
        {
            return await _context.Products.FindAsync(id);
        }

        public async Task AddAsync(Product product)
        {
            await _context.Products.AddAsync(product);
            await _context.SaveChangesAsync();
        }

        public async Task UpdateAsync(Product product)
        {
            _context.Products.Update(product);
            await _context.SaveChangesAsync();
        }

        public async Task DeleteAsync(int id)
        {
            var product = await GetByIdAsync(id);
            if (product != null)
            {
                _context.Products.Remove(product);
                await _context.SaveChangesAsync();
            }
        }
    }
}
```

6 Register Services (Dependency Injection)

Program.cs

```
using Microsoft.EntityFrameworkCore;
using RepoPatternAPI.Data;
```

SHIVA TUTORIAL

Tutorials
C#
ASP.NET CORE MVC
PYTHON
JAVA
JAVASCRIPT
REACT JS
NODE JS
Salesforce
DEVOPS
Software Testing
Selenium JAVA
Selenium Python
Spring MVC & Boot
Angular 21.0

Classroom Training
Online Training

```
using RepoPatternAPI.Repositories;

var builder = WebApplication.CreateBuilder(args);

// Add DbContext
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseInMemoryDatabase("TestDb")); // or SQL Server

// Register Repository
builder.Services.AddScoped<IProductRepository, ProductRepository>();

builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

app.UseSwagger();
app.UseSwaggerUI();

app.MapControllers();

app.Run();
```

7️⃣ Create Controller (REST API)

📁 Controllers/ProductController.cs

```
using Microsoft.AspNetCore.Mvc;
using RepoPatternAPI.Models;
using RepoPatternAPI.Repositories;

namespace RepoPatternAPI.Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class ProductController : ControllerBase
    {
        private readonly IProductRepository _repo;

        public ProductController(IProductRepository repo)
        {
            _repo = repo;
        }

        // GET: api/product
        [HttpGet]
        public async Task<IActionResult> GetAll()
        {
            var data = await _repo.GetAllAsync();
            return Ok(data);
        }

        // GET: api/product/5
        [HttpGet("{id}")]
        public async Task<IActionResult> GetById(int id)
        {
            var product = await _repo.GetByIdAsync(id);
            if (product == null) return NotFound();
            return Ok(product);
        }

        // POST: api/product
        [HttpPost]
        public async Task<IActionResult> Create(Product product)
        {
            await _repo.AddAsync(product);
            return Ok(product);
        }

        // PUT: api/product/5
        [HttpPut("{id}")]
        public async Task<IActionResult> Update(int id, Product product)
        {
            if (id != product.Id) return BadRequest();
```

```
            await _repo.UpdateAsync(product);
            return NoContent();
        }

        // DELETE: api/product/5
        [HttpDelete("{id}")]
        public async Task<IActionResult> Delete(int id)
        {
            await _repo.DeleteAsync(id);
            return NoContent();
        }
    }
}
```

✅ API Endpoints

Method	Endpoint	Description
GET	`/api/product`	**Get all products**
GET	`/api/product/{id}`	**Get by ID**
POST	`/api/product`	**Create**
PUT	`/api/product/{id}`	**Update**
DELETE	`/api/product/{id}`	**Delete**

💡 Why Repository Pattern?

🔥 Pro-Level Improvements (Recommended)

.

Clean separation of concerns

.

Easy unit testing

.

Maintainable code

.

Swappable data source (SQL, MongoDB, etc.)

If you want to make this industry-ready, add:

.

✅ DTOs (Data Transfer Objects)

.

✅ AutoMapper

•

✅ Unit of Work Pattern

•

✅ Global Exception Handling Middleware

•

✅ Logging (Serilog)

•

✅ JWT Authentication

If you want, I can next:

👉 Convert this into complete production architecture (Clean Architecture)
👉 Add JWT Auth + Role-based API
👉 Generate PDF notes for your students (with your logo)



Tags ASP.NET CORE MVC

Facebook Twitter

OLDER
Authentication vs Authorization in ASP.NET CORE

NEWER
Type of Dependancy Injection Example in .NET CORE | Scoped , Transient and Singleton | Type of Services



POSTED BY SHIVA GAUTAM
Hi, I'm Shiva Gautam. With over 15 years of diverse experience in various IT domains, I am now an entrepreneur focusing on both training and software development. My expertise spans across multiple technologies, and I've authored numerous articles on subjects including Java, Python, REACT, NODE, ANGULAR, Cloud, software testing, Django framework, C#, Flutter, Salesforce, Ruby on Rails, .NET, AWS, DEVOPS, AI/ML and more. For additional information, please visit shivaconceptsolution.com and kangaroosoftware.net.

SHIVA TUTORIAL

- Tutorials
- C#
- ASP.NET CORE MVC
- PYTHON
- JAVA
- JAVASCRIPT
- REACT JS
- NODE JS
- Salesforce
- DEVOPS
- Software Testing
- Selenium JAVA
- Selenium Python
- Spring MVC & Boot
- Angular 21.0

- Classroom Training
- Online Training

You may like these posts

ASP.NET CORE MVC

Caching in ASP.NET Core Web API — Complete Tutorial

August 20, 2026

ASP.NET CORE MVC

Entity Framework Core with StoredProcedure Example in Code FIrst And Data First Approach

August 06, 2026

ASP.NET CORE MVC

Best AI Integration Tools in Visual Studio 2022 (2026 Guide)

May 19, 2026

Post a Comment

0 Comments

POST Answer of Questions and ASK to Doubt

✨ JOIN AWS, DEVOPS, SALESFORCE, DATA ANALYTICS, JAVA, .NE

Get 75% Discount Early bird offer CLICK to JOIN 100% Placement Oriented P

Created By Blogger Theme | Distributed By Gooyaabi Templates